Software Supply Chain Analytics: MongoDB-to-PostgreSQL Dimensional Model for PyPI and OSV Data

(Total Words: 1,999)

1. Introduction

This project builds a PostgreSQL analytical model using a software supply chain dataset. The data comes from two sources: PyPI package metadata (pypi_items.json) and OSV vulnerability advisories (vulnerabilities.json). After both raw files are loaded into MongoDB, a Python notebook runs an ETL that reshapes those documents into a five-table Kimball dimensional model in PostgreSQL (ssc schema). PostgreSQL is the final query target, and MongoDB only holds the raw documents. The finished model is validated by four analytical questions about vulnerability exposure, organisational ownership, dependency centrality, and transitive graph risk.

2. Design Rationale

We chose a Kimball dimensional model, built on the star schema pattern, because every analytical question is an aggregation. The questions ask for counts and rankings of advisories, packages, dependencies, and hops, which a star schema answers with simple joins on indexed keys and a GROUP BY. This avoids deep joins across many normalised tables (Kimball & Ross, 2013). This schema has three dimension tables that describe entities (dim_package, dim_version, dim_vulnerability) and two factless fact tables that record relationships (fact_dependency, fact_affected_range). These tables record that a relationship exists, such as a declared dependency or an affected version constraint (Kimball & Ross, 2013).

We use MongoDB only as the raw landing zone. Its flexible storage accepts the deeply nested PyPI and OSV JSON as it arrives, leaving all cleaning to the ETL step (Chodorow, 2013). MongoDB is not where we run the analysis. Document stores like MongoDB make it difficult to aggregate data across multiple entities while maintaining referential integrity (Chodorow, 2013; Elmasri & Navathe, 2016). Conversely, PostgreSQL provides typed columns, foreign keys, unique constraints, and indexes that help ensure data accuracy and consistency (Obe & Hsu, 2017).

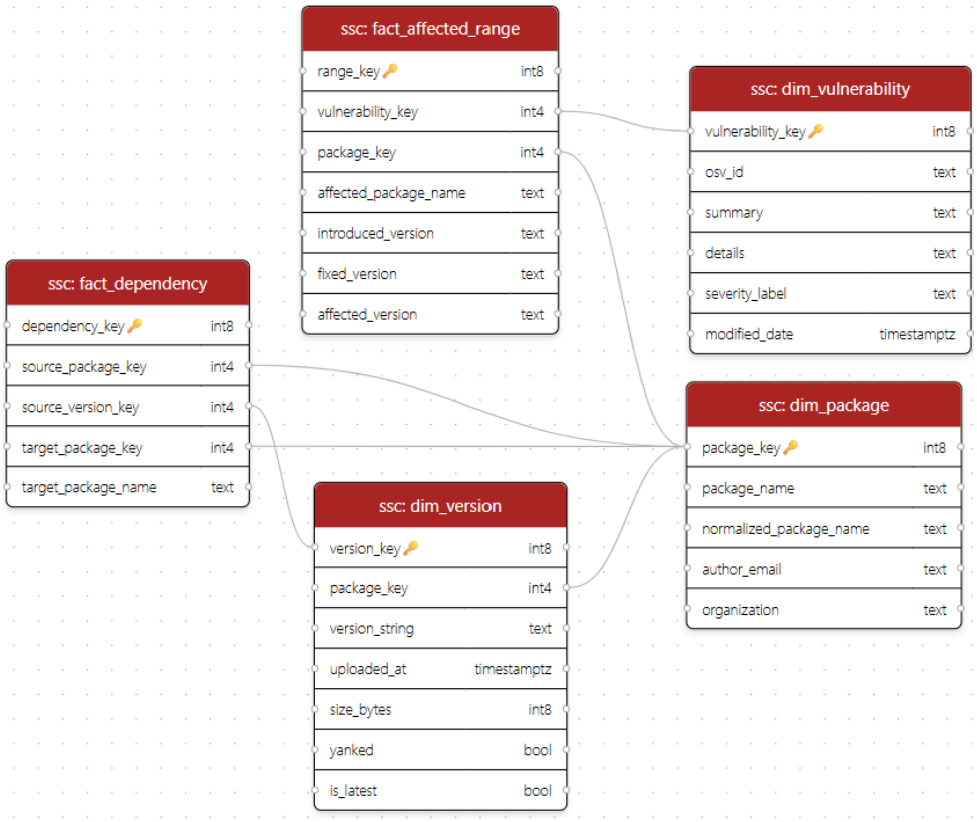
Across the model, we follow Kimball's rule of no NULLs in attributes used for grouping. Missing values are replaced with clear stand-in labels, including No Organisation or UNKNOWN, and every foreign key in a fact table is NOT NULL. When a dependency or advisory points to a package outside our dataset, it references a single Unknown package row (package_key = -1), keeping GROUP BY and COUNT results stable and reproducible (Kimball & Ross, 2013).

3. Schema Overview

The ssc schema has five tables arranged around the shared dim_package dimension (see Figure 1). dim_package holds one row for each package. dim_version holds one row for each published version and links back to its package. fact_dependency records directed dependency edges, meaning one package version depends on a target package. dim_vulnerability holds one row for each distinct OSV advisory, and fact_affected_range connects advisories to the packages and affected version ranges. Both fact tables point back to dim_package. Since they share this one conformed dimension, the full model forms a fact constellation (galaxy schema), and every join is a single step on a surrogate key (Kimball & Ross, 2013).

Figure 1

The ssc Schema



Note. 'fact_dependency' and 'fact_affected_range' are factless facts, and the other three tables are dimensions. Two foreign keys (fact_dependency.target_package_key, fact_affected_range.package_key) resolve to the Unknown member (package_key = -1) when a package falls outside the dataset.

4. Table Grain

The grain of each table is stated before its keys or measures, following Kimball and Ross's (2013) principle that a stable grain prevents aggregations from double-counting or under-counting (see Table 1 for the grain and loaded row counts of each table in the ssc schema).

Table 1
Grain and loaded row counts of the five tables in the ssc schema.

Table	Type	One row represents:	Rows
dim_package	conformed dimension	one PyPI package (identified by its normalised name)	19,989[1]
dim_version	dimension	one published version (release) of one package	1,858,695
fact_dependency	factless fact	one dependency declared by one (source package, source version) on one target package	20,270,815
dim_vulnerability	dimension	one distinct OSV advisory (unique osv_id)	6,308
fact_affected_range	factless fact	one affected-version constraint (a range **or** a pinned version) of one advisory for one package	15,272

Note. Each row gives the grain of one table in the ssc schema, that is, what a single row represents, along with the number of rows loaded by the final ETL run. Counts should be interpreted against these grains when aggregating. The dim_package total includes one Unknown member (package_key = -1), giving 19,989 rather than 19,988.

[1] 19,988 real packages plus one Unknown member (package_key = -1).

Two counting rules come from these grains. A package recurs once per affected range of the same advisory, so advisory counts use COUNT(DISTINCT vulnerability_key), not COUNT(*). Dependency counting faces the same risk twice over: one target may be declared by many versions of a single source, and even a single version's specifier list can repeat a target under different extras or environment markers, so fact_dependency may hold duplicate (source_version, target) rows and dependant counts use COUNT(DISTINCT source_package_key).

5. Keys and Joins

5.1 Primary and Foreign Keys

Every table uses a system-generated BIGSERIAL surrogate primary key (package_key, version_key, dependency_key, vulnerability_key, range_key), which keeps joins independent of business identifiers that may change (Kimball & Ross, 2013). Uniqueness is applied through constraints: dim_package.normalized_package_name and dim_vulnerability.osv_id are UNIQUE; dim_version is unique on (package_key, version_string); and fact_affected_range is unique on (vulnerability_key, affected_package_name, introduced_version, fixed_version, affected_version) with NULLS NOT DISTINCT. For the foreign keys, dim_version.package_key references dim_package; in fact_dependency, source_package_key and target_package_key reference dim_package while source_version_key references dim_version; and in fact_affected_range, vulnerability_key references dim_vulnerability and package_key references dim_package. All fact foreign keys are NOT NULL and use ON DELETE CASCADE; unresolved targets reference the Unknown member (-1) instead of NULL.

5.2 Join Rules and Key Normalisation

All analytical joins are equality joins on surrogate keys, never on raw text, so they use an index and stay unambiguous. The standard paths are dim_version joins dim_package on package_key; fact_dependency joins dim_package on the source and target package_key and joins dim_version on source_version_key; and fact_affected_range joins dim_vulnerability on vulnerability_key and dim_package on package_key.

The one logical join key spanning the two raw sources is the normalised package name. All PyPI names, PEP 508 dependency specifiers and OSV affected package names are normalised in the same way by normalize_package_name(). Any run of hyphen, underscore, or dot becomes a single hyphen, the text is lowercased, and surrounding spaces are removed (Stufft, 2015). Through this, "PyYAML", "pyyaml", and "py_yaml" resolve to one package, and dependency and advisory names match dim_package. Before normalisation, dependency specifiers are first parsed with packaging to remove version constraints and extras (Collins, 2015; Python Packaging Authority, n.d.). A normalised name that does not appear in dim_package is mapped to the Unknown member (-1) to preserve the count.

6. ETL: MongoDB to PostgreSQL

Both JSON files are loaded into MongoDB raw collections, raw_pypi_package and raw_osv_package, preserving nested JSON before transformation into PostgreSQL, the final analytical system. Appendix A provides the raw-field-to-column mappings. Field definitions follow the PyPI JSON API and OSV schema format (Open Source Security Foundation, 2026; PyPI, 2024).

Profiling the MongoDB collections confirms complete release coverage and identifies missing-value issues: 18,221 packages have no organisation and 2,603 OSV advisory entries have no severity label. These checks justify explicit labels rather than NULLs for groupable values.

The PyPI ETL creates package, version and dependency records. Package names are normalised, missing author_email and organization become Not Provided and No Organization and packages are bulk-upserted into dim_package using ON CONFLICT(normalized_package_name). Release entries are flattened into dim_version, with the field-level version rules documented in Appendix A.

Dependency declarations come from _historical_requires_dist, the per-version source of truth for dependency edges. The notebook confirms every version key also appears in releases, confirming zero orphan versions so declarations link safely to dim_version. Each specifier is parsed, normalised, resolved to a package key where possible and inserted into fact_dependency. Since specifiers can include extras, version constraints and environment markers, parsing is handled explicitly (Collins, 2015; Python Packaging Authority, n.d.). Targets outside the released PyPI set are assigned to the Unknown member (package_key = -1).

The OSV ETL deduplicates advisories by osv_id. Each distinct advisory is loaded once into dim_vulnerability, with missing severity stored as UNKNOWN. Affected entries are flattened into fact_affected_range, preferring version ranges then pinned versions and finally a bare advisory-package link where neither exists. Table A5 gives the row-level rules.

The pipeline runs as schema creation, PyPI ETL, OSV ETL and validation because truncating dim_package with CASCADE clears dependent fact tables. Validation confirmed that row counts match Table 1 with zero duplicate affected-range rows, zero Unknown-source dependencies and exactly one latest version per package.

Unused PyPI fields such as info.requires_python, info.license, info.classifiers, info.project_urls, info.yanked, ownership.roles and the urls array as well as OSV fields such as aliases, the CVSS severity array, references and non-severity fields (e.g. cwe_ids), remain in the raw collections, as they were not needed for the analytical questions.

7. Analytical Questions and Results

Four analytical questions were designed to validate that the PostgreSQL schema and ETL process support meaningful analysis of the software supply chain dataset. Table 2 shows which aspect of the model each question tests.

Table 2
Analytical Questions and Their Objectives

Question	Analytical Question	Purpose
Question 1	*Which packages have the most security advisories, and how many are critical, high, or moderate?*	Verifies that vulnerability records aggregate correctly to the package grain while supporting severity-based analysis.
Question 2	*Which GitHub organisations own the most packages, and what percentage of their packages have at least one known vulnerability?*	Verifies that organisation ownership was modelled correctly to support ownership and vulnerability exposure analysis.
Question 3	*Which packages does the whole Python ecosystem rely on most, and do they have known vulnerabilities?*	Verifies that the dependency model correctly supports in-degree analysis and identifies highly depended-upon packages.
Question 4	*Which packages are indirectly exposed through the dependency chain of a known vulnerable package, and how many hops away are they?*	Verifies that recursive dependency traversal is correctly implemented in PostgreSQL to analyse transitive exposure.

Note. The analytical questions were designed to evaluate the correctness of the relational model, ETL process, and PostgreSQL queries by testing package-level aggregation, ownership analysis, dependency analysis, and recursive graph traversal.

7.1 Question 1: Packages with the Most Security Advisories, by Severity

This question requires **dim_package**, **fact_affected_range**, and **dim_vulnerability** to aggregate security advisories by package and severity (see Appendix B.1 for a detailed explanation of the PostgreSQL query).

Table 3
Packages with the Most Security Advisories by Severity

Package	Total Advisories	Critical	High	Moderate	Low	Unknown
tensorflow-gpu	724	6	119	197	105	297
tensorflow-cpu	720	5	115	198	105	297
tensorflow	676	6	122	200	105	243
Django	300	16	64	61	11	148
apache-airflow	209	11	37	60	3	98
Plone	196	5	38	55	2	96
ansible	130	9	24	27	6	64
salt	122	17	31	17	2	55
pillow	119	10	40	12	1	56
mlflow	96	19	41	10	1	25

Note. Total advisories represent distinct vulnerability records associated with each package. A single advisory affecting multiple version ranges is counted once using COUNT(DISTINCT vulnerability_key). Severity categories are based on OSV advisory labels; **Unknown** denotes advisories for which no severity classification is available.

The results in Table 3 indicate that the TensorFlow family (**tensorflow-gpu**: 724, **tensorflow-cpu**: 720, **tensorflow**: 676) has the highest number of distinct advisories, followed by **Django** (300). Furthermore, packages with the most advisories do not necessarily have the most critical vulnerabilities. For instance, the lower-ranked **mlflow** has the highest number of Critical advisories (19). Also, approximately 2,600 advisories are **UNKNOWN**, limiting the completeness of severity-based comparisons. Overall, our findings suggest that our model supports package-level vulnerability analysis.

7.2 Question 2: Organisations by Package Count and Vulnerability Rate

To address this question, we calculated the percentage of each organization's packages linked to at least one known vulnerability using **dim_package** and **fact_affected_range**. The complete PostgreSQL query details are provided in Appendix B.2.

Table 4
Organisations by Package Count and Vulnerability Rate

Organisation	Packages	Packages with Vulnerabilities	Vulnerability Rate
apache-airflow	106	28	26.4%
zope	100	8	8.0%
plone	98	11	11.2%
opentelemetry	84	1	1.2%
jupyter	75	15	20.0%

nvidia	45	0	0.0%
meta-platforms	40	4	10.0%
passagemath	36	0	0.0%
aio-libs	33	4	12.1%
livekit	29	0	0.0%

Note. Vulnerability rate is calculated as the proportion of an organisation's packages with at least one linked advisory (packages with vulnerabilities ÷ total packages × 100). The metric reflects package-level exposure within the released dataset rather than overall organisational security or vulnerability severity.

Our findings in Table 4 show that **apache-airflow** is the organisation with the largest package portfolio (106) and the highest vulnerability rate (26.4%). Conversely, several other large organisations, such as **zope** (100 packages, 8.0%) and **opentelemetry** (84 packages, 1.2%), rank near the top by size but near the bottom by vulnerability rate. These findings imply that the relationship between portfolio size and vulnerability rate lacks consistency across organisations. Overall, the results demonstrate that the model supports organisation-level analysis of package ownership and vulnerability exposure.

7.3 Question 3: Most Directly Depended-Upon Packages

To address this question, we used **fact_dependency** and **fact_affected_range** to identify the most depended-on packages and their known vulnerabilities. The query calculates each package's in-degree by counting distinct dependent packages (see Appendix B.3).

Table 5

Most Directly Depended-Upon Packages and Their Vulnerability Status

Package	Distinct Dependants	Vulnerability Status
pytest	4,972	Has known vulnerability
requests	3,281	Has known vulnerability
numpy	3,029	Has known vulnerability
typing-extensions	2,830	No known vulnerability
pytest-cov	2,560	No known vulnerability
sphinx	2,135	No known vulnerability
six	1,775	No known vulnerability
pydantic	1,768	Has known vulnerability
pandas	1,704	Has known vulnerability
pyyaml	1,703	Has known vulnerability
mypy	1,552	No known vulnerability
black	1,529	Has known vulnerability
setuptools	1,521	Has known vulnerability
Flake8	1,494	No known vulnerability
packaging	1,452	No known vulnerability

Note. Distinct dependants are counted from historical dependency declarations. Vulnerability status indicates the presence of at least one linked advisory in the dataset and does not imply confirmed exploitability or overall package security.

Table 5 shows **pytest** as the most depended-upon package (4,972 distinct dependents), followed by **requests** (3,281), **numpy** (3,029), and **typing-extensions** (2,830). Most packages this high in the ranking carry at least one known advisory. **typing-extensions** is the exception: fourth by dependent count, yet with no linked advisories. Overall, our model supports the identification of packages with the greatest potential ecosystem impact based on historical dependency relationships.

7.4 Question 4: Transitive Exposure from a Vulnerable Package

For this bonus question, we analysed transitive dependency exposure using **fact_dependency**, **dim_version**, and **dim_package**. A recursive CTE traverses the latest-version dependency graph to identify direct and indirect dependents (see Appendix B.4).

As shown in Table 6, a total of 13,590 packages are reachable from **numpy** within eight dependency hops. Exposure peaks at three hops (3,370 packages) and four hops (2,415) before declining sharply, placing the bulk of the reachable set several steps away rather than at its immediate boundary. Additionally, the 2,661 direct dependents, such as **3lc** and **AeroSandbox**, constitute less than one-fifth of that total closure (see Table 7). In conclusion, these findings suggest that a serious vulnerability in widely used runtime libraries can have a broad structural impact across the ecosystem.

Table 6

Transitive Exposure from numpy by Shortest Dependency Distance

Shortest Hop Distance	Reachable Packages
1	2,661
2	2,276
3	3,370
4	2,415
5	1,843
6	826
7	193
8	6

Note. Total of 13590 reachable packages. 8 hops is the maximum depth reached.

Table 7
Sample of Packages Reached from numpy by Shortest Dependency Distance

Package	Shortest Hop Distance
3lc	1
AeroSandbox	1
AllanTools	1
BabelDOC	1
Boruta	1
Bottleneck	1
Brian2	1
CMGDB	1
Cartopy	1
CausalPy	1
ConfigSpace	1
CoolProp	1
Crawl4AI	1
CyRK	1
DIPEX	1

Note. This table presents only the first **15** packages returned by the recursive query (LIMIT 15). All listed packages are **direct dependants** of numpy (shortest hop distance = 1) and are shown for illustrative purposes. The complete transitive exposure summary is reported in **Table 6.**

8. Assumptions, Exclusions and Caveats

The model is not based on the whole PyPI ecosystem, so all counts are dataset-specific.

A straightforward loader can fail at MongoDB's 16MiB BSON document-size limit, since large release histories or dependency declarations can become oversized when stored as one nested document (MongoDB, 2024). The relational model reduces this by flattening nested arrays into package, version, dependency and affected-range rows.

Dependency parsing is based on _historical_requires_dist. The model therefore records declared relationships rather than installation-specific outcomes (Collins, 2015; Python Packaging Authority, n.d.) and dependency analysis counts distinct packages rather than raw rows.

Vulnerability counts reflect OSV advisory linkage and affected-version coverage, not confirmed exploitability. Some OSV ranges express their upper bound with last_affected, which the ETL does not map; these are treated as open-ended, which may slightly overstate range-level coverage but does not affect advisory counts. Because one advisory can generate multiple affected-range rows, advisory-level queries use COUNT(DISTINCT vulnerability_key). Real-package queries exclude the Unknown member, so counts should be read with each table's grain and filters in mind.

Advisory counts and organisation-level vulnerability rates are not direct security measures, as popular packages may receive more scrutiny and organisation-level rates show only the share of packages linked to known advisories, not overall security posture.

The graph extension uses latest-version dependencies only, so historical paths are not fully represented. numpy is the root because it is a verified high-dependency vulnerable package. The query uses a recursive CTE with a materialised edge set and targets PostgreSQL 15+, as the schema uses UNIQUE NULLS NOT DISTINCT (PostgreSQL Global Development Group, 2025a, 2025b).

9. Contribution Summary

Initially, all members contributed to the schema and relational model design. Later, Sujin and Idil Birsu drafted independent versions of the ETL pipeline, which the group then evaluated. Following this, to validate the final implementation, each member proposed and implemented one analytical question. Finally, Lara and Seda constructed the LLM prompt document. Detailed individual contributions are listed in Table 8.

Table 8
Summary of Individual Contributions

Member	Question Proposed and Implemented	Other Individual Contributions
Sujin Baek	**Q1** — Packages with the most advisories, by severity	Developed an initial ETL pipeline draft; wrote Sections **1–3** (Introduction, Design Rationale, Schema Overview).
Seda Kart	**Q2** — Organizations by package count and vulnerability rate	Co-authored the LLM prompt document; wrote Sections **4–6.1** (Table Grain, Keys & Joins, Data Dictionary), Appendix A.
Lara Gunseli Safak	**Q3** — Most depended-upon packages (in-degree)	Co-authored the LLM prompt document; wrote Sections **6** (ETL Process) and **8** (Assumptions, Exclusions & Caveats).
Idil Birsu Balaban	**Q4 — Transitive exposure from a vulnerable package (bonus)**	Developed an initial ETL pipeline draft; wrote Sections 7 (Analytical Questions), 9 (Contribution Summary), AI Statement and Appendix B.

Note. All team members jointly designed the data model and validated the final implementation. ETL development was initiated by **Sujin Baek** and **Idil Birsu Balaban**, the LLM prompt document was co-authored by **Lara Gunseli Safak** and **Seda Kart**, and each member proposed and implemented one analytical question. Report writing responsibilities are summarised in the table above.

AI Declaration

Initially, LLM tools, namely **Claude** and **ChatGPT**, were used as assistants within a human-in-the-loop process to align with the project brief. These assistants were used for drafting SQL queries, scaffolding the ETL notebook code, providing wording suggestions, and debugging errors. All design decisions, such as the star-schema model, table grains, the Unknown-member (-1) convention, key normalisation, and the analytical question choices, were made and owned by the team members.

All AI-generated content was reviewed by the team before implementation. The schema and ETL were verified by running the full pipeline end-to-end and cross-checking the outputs against the source data.

None of the team members gave AI tools access to produce the required files or submit the assignment on behalf of the team. All figures reported in the document were produced by running the final queries against our PostgreSQL database and were not taken from AI output.

References

Chodorow, K. (2013). MongoDB: The definitive guide: Powerful and scalable data storage (2nd ed.). O'Reilly Media.

Collins, R. (2015). *PEP 508 – Dependency specification for Python software packages*. Python Software Foundation. https://peps.python.org/pep-0508/

Elmasri, R., & Navathe, S. (2016). Fundamentals of database systems (7th ed.). Pearson.

Kimball, R., & Ross, M. (2013). The data warehouse toolkit: The definitive guide to dimensional modeling (3rd ed.). Wiley.

MongoDB. (2024). GridFS for self-managed deployments. MongoDB Manual. https://www.mongodb.com/docs/manual/core/gridfs/

Obe, R. O., & Hsu, L. S. (2017). PostgreSQL: Up and running: A practical guide to the advanced open source database (3rd ed.). O'Reilly Media.

Open Source Security Foundation. (2026). Open Source Vulnerability format. https://ossf.github.io/osv-schema/

PostgreSQL Global Development Group. (2025a). Unique indexes. PostgreSQL Documentation. https://www.postgresql.org/docs/current/indexes-unique.html

PostgreSQL Global Development Group. (2025b). WITH queries: Common table expressions. PostgreSQL Documentation. https://www.postgresql.org/docs/current/queries-with.html

PyPI. (2024). *PyPI JSON API documentation*. https://docs.pypi.org/api/json/

Python Packaging Authority. (n.d.). Dependency specifiers. Python Packaging User Guide. Retrieved July 6, 2026, from https://packaging.python.org/specifications/dependency-specifiers/

Stufft, D. (2015). *PEP 503 – Simple repository API*. Python Software Foundation. https://peps.python.org/pep-0503/

Appendices

Appendix A: Data Dictionary

The dictionary maps the two raw collections (raw_pypi_package, raw_osv_package) to the five relational tables (Tables A1 to A5). PyPI source fields follow the PyPI JSON API (PyPI, 2024) and vulnerability fields follow the OSV schema (Open Source Security Foundation, 2026); package name normalisation follows PEP 503 (Stufft, 2015). Fields not needed by any analytical question, for example requires_python, license, classifiers, project_urls and ownership.roles on the PyPI side, and aliases, the CVSS severity array, references and cwe_ids on the OSV side, were left in the raw MongoDB collections and are not mapped below.

Table A1

Field mapping for dim_package (grain: one PyPI package).

PostgreSQL column	Type	Raw source field	Transformation or notes
package_key	BIGSERIAL PK	not applicable	System generated surrogate; key -1 is reserved for the Unknown member
package_name	TEXT NOT NULL	info.name (fallback package_name)	The original name, before normalisation
normalized_package_name	TEXT NOT NULL UNIQUE	info.name	Runs of hyphen, underscore, or dot become a single hyphen, then the text is lowercased and trimmed (Stufft, 2015); used as the natural key
author_email	TEXT NOT NULL	info.author_email	If null or empty, set to ‘Not Provided’
organization	TEXT NOT NULL	ownership.organization	If null or empty, set to ‘No Organization’

Note. This table maps fields from the raw_pypi_package collection to dim_package, the conformed package dimension (grain: one PyPI package). package_key is a system generated surrogate, with key -1 reserved for the Unknown member used when a referenced package falls outside the dataset. Package names are normalised following PEP 503 (Stufft, 2015), and missing author_email and organization values are replaced with the labels 'Not Provided' and 'No Organization' so they remain usable for grouping. PK = primary key.

Table A2

Field mapping for dim_version (grain: one published version of a package).

PostgreSQL column	Type	Raw source field	Transformation or notes
version_key	BIGSERIAL PK	not applicable	System generated surrogate
package_key	INT NOT NULL FK	(resolved)	References dim_package, matched by normalised name
version_string	TEXT NOT NULL	key of the releases dictionary	The version label
uploaded_at	TIMESTAMPTZ NULL	releases.<ver>[0].upload_time_iso_8601	Taken from the first release file object
size_bytes	BIGINT NULL	releases.<ver>[0].size	Taken from the first release file object
yanked	BOOLEAN	releases.<ver>[*].yanked	TRUE if any file object is yanked
is_latest	BOOLEAN	derived	TRUE where version_string equals info.version

Note. This table maps release data from the raw_pypi_package collection to dim_version (grain: one published version of a package). package_key is a foreign key resolving to dim_package by normalised name. uploaded_at and size_bytes are read from the first release file object of each version; yanked is TRUE if any file object for that version is yanked; and is_latest is derived, set to TRUE where the version string equals info.version. FK = foreign key.

Table A3

Field mapping for fact_dependency (grain: one dependency declared by a source package version).

PostgreSQL column	Type	Raw source field	Transformation or notes
dependency_key	BIGSERIAL PK	not applicable	System generated surrogate
source_package_key	INT NOT NULL FK	(resolved)	Declaring package; references dim_package
source_version_key	INT NOT NULL FK	(resolved)	Declaring version; references dim_version
target_package_key	INT NOT NULL FK	(resolved)	Target package; references dim_package, or key -1 if outside the dataset
target_package_name	TEXT NOT NULL	_historical_requires_dist.<ver>[]	Bare name obtained with packaging. Requirement (Collins, 2015), then normalised

Note. This table maps historical dependency declarations (_historical_requires_dist) from the raw_pypi_package collection to fact_dependency (grain: one dependency declared by one source package version on one target). Each specifier is parsed with packaging.Requirement following PEP 508 (Collins, 2015) to obtain the bare package name, which is then normalised (Stufft, 2015). Targets outside the released dataset resolve to the Unknown member (key -1) rather than being dropped, so dependant counts stay complete.

Table A4

Field mapping for dim_vulnerability (grain: one distinct OSV advisory).

PostgreSQL column	Type	Raw source field	Transformation or notes
vulnerability_key	BIGSERIAL PK	not applicable	System generated surrogate
osv_id	TEXT NOT NULL UNIQUE	vulnerabilities[].id	Natural key; duplicates collapsed from 7,297 to 6,308
summary	TEXT NULL	vulnerabilities[].summary	Stored as is
details	TEXT NULL	vulnerabilities[].details	Markdown advisory text
severity_label	TEXT NOT NULL	vulnerabilities[].database_specific.severity	If null or empty, set to ‘UNKNOWN’
modified_date	TIMESTAMPTZ NULL	vulnerabilities[].modified	Cast to timestamptz

Note. This table maps advisory records from the raw_osv_package collection to dim_vulnerability (grain: one distinct OSV advisory). Advisories are deduplicated by osv_id, reducing 7,297 raw entries to 6,308 distinct advisories. Missing severity labels are stored as 'UNKNOWN' so that severity-based grouping remains complete.

Table A5

Field mapping for fact_affected_range (grain: one affected version rule of an advisory for a package).

PostgreSQL column	Type	Raw source field	Transformation or notes
range_key	BIGSERIAL PK	not applicable	System generated surrogate
vulnerability_key	INT NOT NULL FK	(resolved)	References dim_vulnerability, matched by osv_id
package_key	INT NOT NULL FK	(resolved)	Affected package; references dim_package, or key -1 if outside the dataset
affected_package_name	TEXT NOT NULL	vulnerabilities[].affected[].package.name	Normalised (Stufft, 2015)
introduced_version	TEXT NULL	affected[].ranges[].events[].introduced	Lower bound of a range; one row per introduced/fixed pair — a single OSV range can contain several pairs (one per affected release branch)
fixed_version	TEXT NULL	affected[].ranges[].events[].fixed	Upper bound of a range; NULL with introduced_version set means no fix in that branch (open-ended)
affected_version	TEXT NULL	affected[].versions[]	Used when no range is present; one row per pinned version. If neither ranges nor versions exist, a bare link row is kept with all three version columns NULL

Note. This table maps the affected entries of OSV advisories (**raw_osv_package**) to **fact_affected_range** (grain: one affected version rule of an advisory for one package). Each row records either a version range (introduced/fixed bounds) or a single pinned version; **affected_version** is used only when no range is present. **affected_package_name** is normalised (Stufft, 2015), and packages outside the dataset resolve to the Unknown member (key -1).

Appendix B: PostgreSQL Queries for Analytical Questions

Here we present the queries used for the four analytical questions discussed in Section 7. Each excludes the Unknown member (-1) where relevant and uses distinct counts to avoid double-counting.

B.1 Q1 – Packages with the Most Security Advisories by Severity

Model & Query Description

The query joins **dim_package**, **fact_affected_range**, and **dim_vulnerability**. **dim_package** holds one row per package, **dim_vulnerability** stores advisory IDs and severity labels, and **fact_affected_range** links the two by recording which advisories affect which packages. Counting **DISTINCT vulnerability_key** prevents double-counting when a single advisory spans multiple affected version ranges. Conditional aggregation then splits the total into Critical, High, Moderate, Low, and Unknown severity buckets per package.

SQL Query

```
SELECT
  dp.package_name,
  COUNT(DISTINCT v.vulnerability_key) AS total_advisories,
  COUNT(DISTINCT CASE WHEN UPPER(v.severity_label) = 'CRITICAL'
                 THEN v.vulnerability_key END) AS critical_count,
  COUNT(DISTINCT CASE WHEN UPPER(v.severity_label) = 'HIGH'
                 THEN v.vulnerability_key END) AS high_count,
  COUNT(DISTINCT CASE WHEN UPPER(v.severity_label)
                    IN ('MODERATE', 'MEDIUM')
                 THEN v.vulnerability_key END) AS moderate_count,
  COUNT(DISTINCT CASE WHEN UPPER(v.severity_label) = 'LOW'
                 THEN v.vulnerability_key END) AS low_count,
  COUNT(DISTINCT CASE WHEN UPPER(v.severity_label)
                    NOT IN ('CRITICAL','HIGH','MODERATE','MEDIUM','LOW')
                 THEN v.vulnerability_key END) AS unknown_count
```

```
FROM ssc.dim_package dp
JOIN ssc.fact_affected_range far ON far.package_key = dp.package_key
JOIN ssc.dim_vulnerability v ON v.vulnerability_key = far.vulnerability_key
WHERE dp.package_key <> -1 -- exclude the Unknown member
GROUP BY dp.package_name
ORDER BY total_advisories DESC
LIMIT 10;
```

B.2 Q2 – Organisations by Package Count and Vulnerability Rate

Model & Query Description

The query for the second question joins **dim_package** and **fact_affected_range**. While **dim_package** consists of package ownership information, **fact_affected_range** identifies packages linked to known advisories. Packages are grouped by organisation, excluding the placeholder **"No Organization"** and the Unknown member (-1). A **LEFT JOIN** to a distinct set of vulnerable packages flags whether each package carries at least one advisory, and the vulnerability rate follows as the share of flagged packages within each organisation. The **HAVING COUNT(*) >= 5** filter drops organisations with fewer than five packages, where the resulting percentages would be too unstable to interpret.

SQL Query

```
SELECT
  dp.organization,
  COUNT(*) AS total_packages,
  COUNT(*) FILTER (WHERE vuln.package_key IS NOT NULL) AS vulnerable_packages,
  ROUND(
    100.0 * COUNT(*) FILTER (WHERE vuln.package_key IS NOT NULL)
    / NULLIF(COUNT(*), 0),
    1
  ) AS vulnerable_pct
FROM ssc.dim_package dp
```

```
LEFT JOIN (
   -- one row per real package that appears anywhere in the advisory bridge
   SELECT DISTINCT package_key
   FROM ssc.fact_affected_range
   WHERE package_key <> -1
) vuln ON vuln.package_key = dp.package_key
WHERE dp.organization <> 'No Organization'  -- Kimball surrogate for missing org
  AND dp.package_key  <> -1 -- exclude the Unknown member
GROUP BY dp.organization
HAVING COUNT(*) >= 5
ORDER BY total_packages DESC
LIMIT 10;
```

B.3 Q3 – Most Depended-Upon Packages and Their Vulnerability Status

Model & Query Description

The query joins the fact_dependency and fact_affected_range tables. fact_dependency consists of directed dependency relationships, and fact_affected_range identifies packages linked to known advisories. In-degree is calculated as the count of distinct source packages per dependency target. A LEFT JOIN to a distinct set of vulnerable packages then classifies each target as having a known vulnerability, no known vulnerability, or falling outside the dataset. Later, it groups by the target package name rather than the target package key, which preserves out-of-dataset targets that share the Unknown member, keeping the dependency counts accurate.

SQL Query

```
WITH dependents AS (
   SELECT
      fd.target_package_name,
      MAX(fd.target_package_key) AS target_package_key,
      COUNT(DISTINCT fd.source_package_key) AS dependent_count
```

```
    FROM ssc.fact_dependency fd
    GROUP BY fd.target_package_name
)
SELECT
    d.target_package_name,
    d.dependent_count,
    CASE
        WHEN d.target_package_key = -1  THEN 'target not in dataset'
        WHEN v.package_key IS NOT NULL  THEN 'has known vulnerability'
        ELSE 'no known vulnerability'
    END AS vulnerability_status
FROM dependents d
LEFT JOIN (
    SELECT DISTINCT package_key
    FROM ssc.fact_affected_range
    WHERE package_key <> -1
) v ON v.package_key = d.target_package_key
ORDER BY d.dependent_count DESC
LIMIT 15;
```

B.4 Q4 – Transitive Exposure from a Vulnerable Package

Model & Query Description

The query draws on **fact_dependency** for the dependency graph edges, **dim_version** to restrict those edges to each package's latest version, and **dim_package** to locate the root. A preliminary query ranks vulnerable packages by in-degree using the same graph (Table B.4.1). **pytest** records the highest in-degree, but **numpy** was chosen as the root because it is a runtime dependency with multiple known advisories, whereas **pytest** is used almost exclusively for testing and rarely ships in production code. **numpy** is therefore the more meaningful starting point for transitive exposure analysis.

The recursive CTE builds a set of distinct latest-version edges, dropping self-loops and targets outside the dataset. It walks the graph backwards, hop by hop, to collect indirect dependents starting from **numpy**'s direct dependents. To deduplicate paths at each step, **UNION** is used rather than **UNION ALL**, which also prevents cycles from causing infinite recursion. A depth cap of twenty acts as a safety guard, though the traversal naturally terminates at eight hops. A companion query then groups the reachable packages by shortest hop distance to show how exposure spreads across the graph.

Table B.4.1.
Vulnerable Packages Ranked by In-Degree (Q4 Pre-check)

Package	In-degree
pytest	3,915
numpy	2,661
requests	2,630
pydantic	1,546
pandas	1,457
PyYAML	1,446
scipy	1,002
httpx	930
tqdm	835
black	835
aiohttp	789
pillow	777
setuptools	740
Jinja2	727
torch	620

Note. In-degree is calculated as the number of distinct dependant packages. This pre-check informed the selection of **numpy** as the root package for the recursive graph query, rather than pytest, because numpy is a more representative runtime dependency for supply-chain exposure analysis.

Pre-test SQL Query

```
WITH edges AS MATERIALIZED (
  SELECT DISTINCT d.source_package_key AS src,
         d.target_package_key AS dst
  FROM ssc.fact_dependency d
  JOIN ssc.dim_version v
   ON v.version_key = d.source_version_key
```

```
    AND v.is_latest
   WHERE d.target_package_key <> -1                -- real targets only
     AND d.source_package_key <> d.target_package_key   -- no self-loops
),
vulnerable_packages AS (
   -- distinct real packages that carry at least one advisory
   SELECT DISTINCT package_key
   FROM ssc.fact_affected_range
   WHERE package_key <> -1
)
SELECT
   dp.package_name,
   COUNT(DISTINCT e.src) AS indegree
FROM edges e
JOIN ssc.dim_package      dp ON dp.package_key = e.dst
JOIN vulnerable_packages  vp ON vp.package_key = e.dst
GROUP BY dp.package_name
ORDER BY indegree DESC
LIMIT 15;
```

Recursive Graph SQL Query

```
WITH RECURSIVE
edges AS MATERIALIZED (
   -- Distinct, latest-version, real-target adjacency (the graph's edge set).
   SELECT DISTINCT d.source_package_key AS src,
                   d.target_package_key AS dst
   FROM ssc.fact_dependency d
   JOIN ssc.dim_version v
```

```
     ON v.version_key = d.source_version_key
    AND v.is_latest
   WHERE d.target_package_key <> -1                -- real targets only
     AND d.source_package_key <> d.target_package_key   -- no self-loops
),
exposed(pkg, depth) AS (
   -- Anchor: direct dependents of the root (1 hop away).
   SELECT src, 1
   FROM edges
   WHERE dst = (
      SELECT package_key FROM ssc.dim_package
      WHERE normalized_package_name = 'numpy'
   )

   UNION   -- UNION (not UNION ALL) dedups automatically -> cycle-safe

   -- Recursive step: follow edges BACKWARDS — dependents of an exposed package.
   SELECT e.src, x.depth + 1
   FROM exposed x
   JOIN edges e ON e.dst = x.pkg
   WHERE x.depth < 20                          -- defensive depth guard
)
SELECT
   p.package_name,
   MIN(x.depth) AS shortest_hops
FROM exposed x
JOIN ssc.dim_package p ON p.package_key = x.pkg
```

```
GROUP BY p.package_name
ORDER BY shortest_hops, p.package_name
LIMIT 15;
```

Companion Query: Exposure by Distance

```
WITH RECURSIVE
edges AS MATERIALIZED (
SELECT DISTINCT d.source_package_key AS src,
d.target_package_key AS dst
FROM ssc.fact_dependency d
JOIN ssc.dim_version v
ON v.version_key = d.source_version_key
AND v.is_latest
WHERE d.target_package_key <> -1
AND d.source_package_key <> d.target_package_key
),
exposed(pkg, depth) AS (
SELECT src, 1
FROM edges
WHERE dst = (
SELECT package_key FROM ssc.dim_package
WHERE normalized_package_name = 'numpy'
)

UNION

SELECT e.src, x.depth + 1
```

```
FROM exposed x
JOIN edges e ON e.dst = x.pkg
WHERE x.depth < 20
)
SELECT
shortest.depth AS shortest_hops,
COUNT(*) AS packages_at_this_distance
FROM (
SELECT pkg, MIN(depth) AS depth
FROM exposed
GROUP BY pkg
) shortest
GROUP BY shortest.depth
ORDER BY shortest_hops;
```